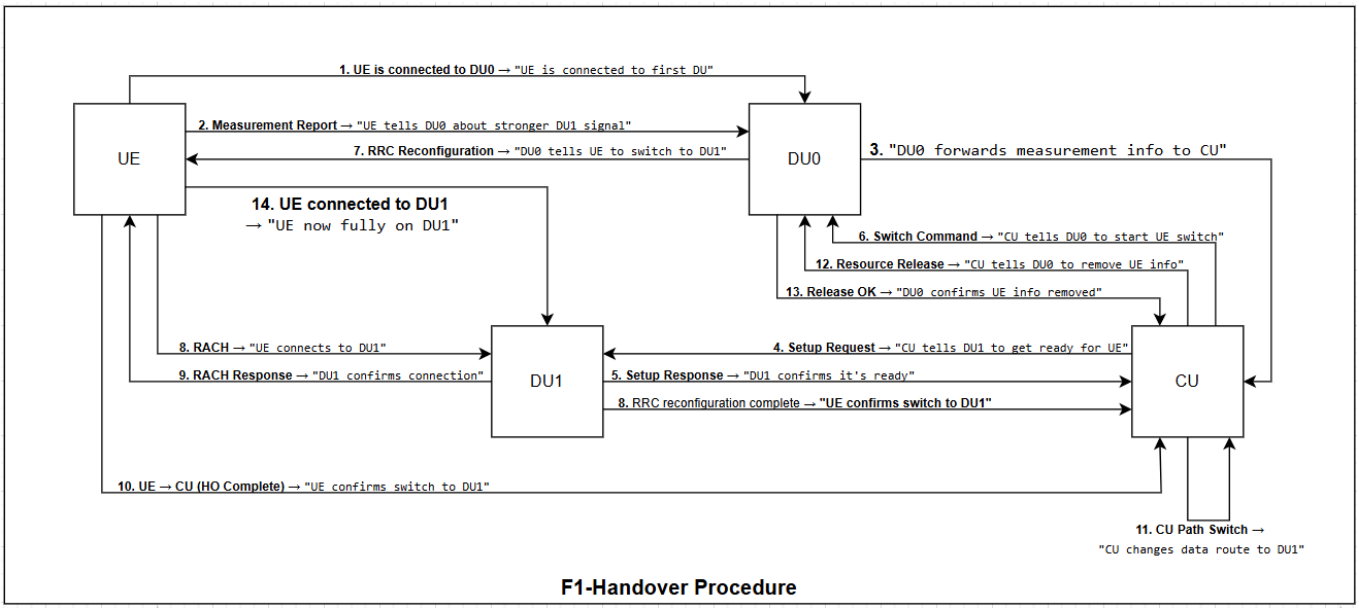
F1 Handover Guide of 5G Setup with OAI-RAN and Open5GS on GCP

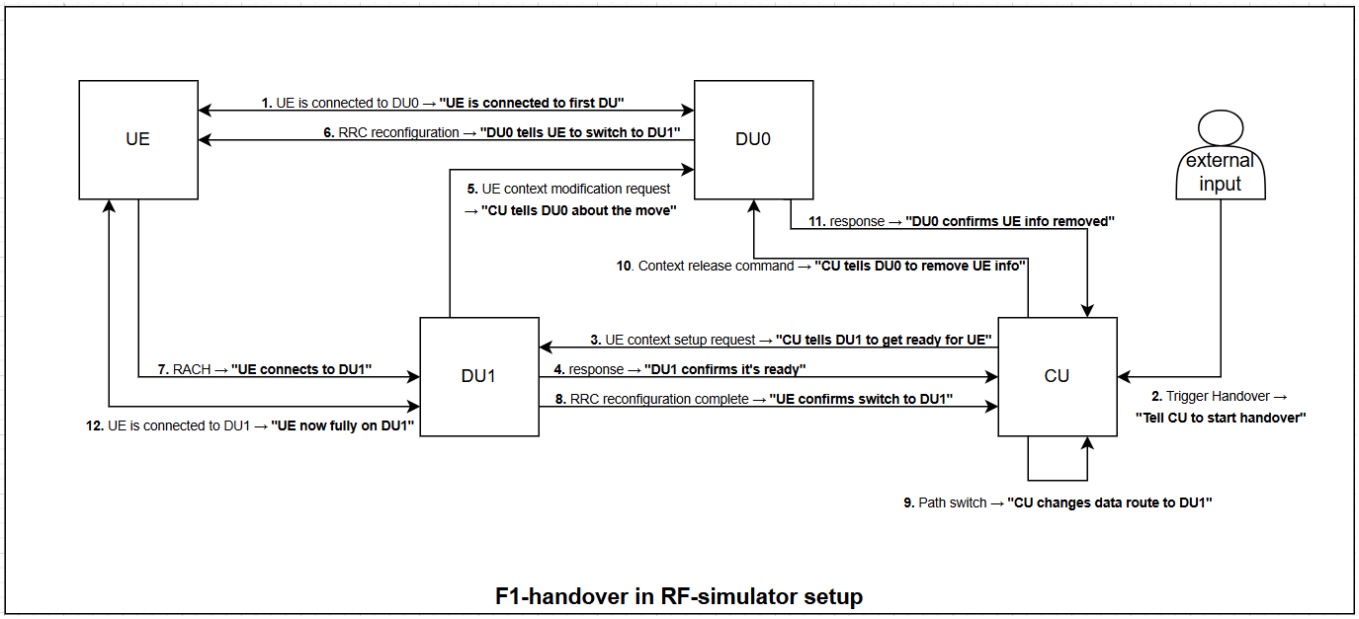
F1 Handover Guide of 5G Setup with OAI-RAN and Open5GS on GCP

This guide provides a complete, step-by-step walkthrough for deploying a virtualized 5G Standalone (SA) network on Google Cloud Platform (GCP). It uses OpenAirInterface (OAI) for the Radio Access Network (RAN) and Open5GS for the 5G Core (5GC).

The final architecture will consist of a consolidated OAI CU-CP/CU-UP, two OAI DUs, and a simulated OAI UE, configured to demonstrate a successful F1 handover between the two DUs.



F1-Handover Procedure



F1-handover in RF-simulator setup

Part 1: GCP Environment and Cost Management

1.1. Leveraging the GCP Free Trial

Google Cloud offers a 90-day, $300 free trial for new users, which is more than sufficient to cover all costs for this deployment.

- **Machine Types:** We will use a mix of `e2-small` and `n2-standard-4` VMs. The OAI RAN components require the N2 series for critical CPU instruction set support (AVX-512).
- **Cost Estimation:** With the $300 credit, the cost for this entire setup during the trial period will be **$0**.
- ⚠️ **Important: Cleanup!** To avoid any charges after the trial ends, you **must** shut down and delete your created resources as outlined in the final section of this guide.

1.2. Initial Project Setup

1. Navigate to the Google Cloud Console.
2. Create a new GCP project. Give it a memorable name like `5g-f1-handover-proj`.
3. Ensure that billing is enabled for the project by linking your free trial billing account.

1.3. VPC Network and Firewall Configuration

1. **Create a VPC:**
   - Navigate to **VPC network > VPC networks**.
   - Click **CREATE VPC NETWORK**.

- **Name:** `oai-5g-vpc`
- **Subnet creation mode:** Custom
- **New subnet:**
  - Name: `oai-subnet`
  - Region: `us-central1` (or another cost-effective US region)
  - IP address range: `172.17.0.0/24`
- Click **Create**.

2. **Configure Firewall Rules:**
   - Navigate to **VPC network > Firewall**.
   - **Rule 1: Allow SSH (for management)**
     - Click **CREATE FIREWALL RULE**.
     - **Name:** `allow-ssh`
     - **Network:** `oai-5g-vpc`
     - **Targets:** All instances in the network
     - **Source filter:** IPv4 ranges
     - **Source IPv4 ranges:** `0.0.0.0/0` (allows SSH from any IP)
     - **Protocols and ports:** Specified protocols and ports > `tcp:22`
     - Click **Create**.
   - **Rule 2: Allow Internal Communication**
     - Click **CREATE FIREWALL RULE**.
     - **Name:** `allow-internal-all`
     - **Network:** `oai-5g-vpc`
     - **Targets:** All instances in the network
     - **Source filter:** IPv4 ranges
     - **Source IPv4 ranges:** `172.17.0.0/24` (the range of our subnet)
     - **Protocols and ports:** Allow all
     - Click **Create**.
3. **Configure Cloud NAT for Internet Access:**
   To allow the UE to access the internet through the 5G core, we'll use a Cloud NAT instead of `iptables`.
   - Navigate to **Network services > Cloud NAT**.
   - Click **GET STARTED** or **CREATE NAT GATEWAY**.
   - **Gateway name:** `oai-5g-nat`
   - **VPC network:** `oai-5g-vpc`
   - **Region:** `us-central1`
   - **Cloud Router:** Click **Create new router**, name it `oai-5g-router`, and click **Create**.
   - Leave other settings as default and click **Create**.

Part 2: Provisioning Virtual Machines

Create five VMs with static internal IPs.

1. **Reserve Static Internal IPs:**
   - Navigate to **VPC network > IP addresses** and click **RESERVE INTERNAL STATIC IP ADDRESS** five times for the following IPs:
     - `172.17.0.91` (for `oai-nr-ue-vm`)
     - `172.17.0.92` (for `oai-du0-vm`)
     - `172.17.0.93` (for `oai-cu-vm`)
     - `172.17.0.94` (for `oai-du1-vm`)
     - `172.17.0.95` (for `open5gs-vm`)
2. **Create the Virtual Machines:**

   - Navigate to **Compute Engine > VM instances** and click **CREATE INSTANCE** for each VM.
   - **VM Creation: Open5GS Core (`open5gs-vm`)**
     - **Name:** `open5gs-vm`
     - **Machine type:** `e2-small`
     - **Boot disk:** Ubuntu 24.04 LTS, 30 GB
     - Under **Advanced options > Networking**:
       - **Network:** `oai-5g-vpc`
       - **Primary internal IP:** The reserved `172.17.0.95`.
       - **External IPv4 address:** Ephemeral
       - Check **Enable IP forwarding**.
   - **VM Creation: All OAI RAN VMs**
     - Create the four OAI VMs (`oai-cu-vm`, `oai-du1-vm`, `oai-du0-vm`, `oai-nr-ue-vm`) with the following settings:
     - **Name:** The respective VM name from the table below.
     - **Series: N2** (Critical for AVX-512 support)
     - **Machine type: n2-standard-4** (4 vCPU, 16 GB memory)
     - **Boot disk:** Ubuntu 24.04 LTS, 30 GB
     - Under **Advanced options > Networking**:
       - **Network:** `oai-5g-vpc`
       - **Primary internal IP:** Select the corresponding reserved IP from the table.
       - **External IPv4 address:** Ephemeral (for SSH access).

VM Roles and IPs:

Final Role	GCP VM Name	Reserved Static IP
Open5GS Core	`open5gs-vm`	`172.17.0.95`
OAI CU-CP & CU-UP	`oai-cu-vm`	`172.17.0.93`
OAI Source DU (DU0)	`oai-du0-vm`	`172.17.0.92`
OAI Target DU (DU1)	`oai-du1-vm`	`172.17.0.94`
OAI NR-UE (RF Server)	`oai-nr-ue-vm`	`172.17.0.91`

Part 3: Setting up SSH for VS Code and Google Cloud

This guide details the steps required to securely connect to your Google Cloud (GCP) Virtual Machines (VMs) using Visual Studio Code (VS Code) and an SSH key pair. This method is more secure than using passwords and provides a powerful, integrated development environment.

3.1. Create SSH Key Pairs

The first step is to generate a secure key pair on your local computer. This consists of a **public key** (which you share with GCP) and a **private key** (which you keep secret on your machine).

1. **Open PowerShell or Terminal** on your local Windows, Mac, or Linux machine.
2. Run the `ssh-keygen` command. We will specify the RSA algorithm and a key size of 4096 bits for strong security and username. This username will be created on the GCP VMs you connect to.

```
ssh-keygen -t rsa -b 4096 -C "telcomaan"
```

3. The command will prompt you for the following:
   - `Enter file in which to save the key...` : It is highly recommended to **press Enter** to accept the default file location and name (`id_rsa`). This is the standard location that SSH tools look for keys.
   - `Enter passphrase (empty for no passphrase):` : Type a strong, memorable password and press Enter. This is a crucial security step that encrypts your private key. Even if someone steals your private key file, they cannot use it without this passphrase.
   - `Enter same passphrase again:` : Re-type the exact same password and press Enter.

4. **Confirm Key Creation:** Once successful, the command will confirm that it has saved your "identification" (private key) and "public key" in your `.ssh` directory.
    - **Private Key:** `~/.ssh/id_rsa`
    - **Public Key:** `~/.ssh/id_rsa.pub`

3.2. Add the Public Key on GCP

Now, you must provide your public key to Google Cloud. By adding it as project-wide metadata, you grant access to all VMs within that project.

1. **Display and Copy Your Public Key:** In your local terminal, run the following command to display the contents of your public key file.

```
cat ~/.ssh/id_rsa.pub
```

2. The output will be a long, single line of text starting with `ssh-rsa`. **Select this entire line of text and copy it** to your clipboard.
3. **Navigate to GCP Metadata:**
    - In the Google Cloud Console, use the navigation menu (☰) to go to **Compute Engine > Metadata**.
4. **Add the SSH Key:**
    - Select the **SSH Keys** tab.
    - Click the **ADD SSH KEY** button.
    - Paste your copied public key into the large text box that appears.
    - GCP will automatically parse the username from the end of the key.
    - Click **Save**.

Within a few minutes, Google Cloud will distribute this key to all VMs in the project, authorizing access for your private key.

3.3. Install and Set Up Remote SSH in VS Code

The final step is to configure VS Code to use your private key to establish a connection.

1. **Copy the External IP:** Ensure the VM you want to connect to has a public IP address.
    - In the GCP Console, go to **Compute Engine > VM instances**.
    - Copy the External IP
2. **Install the "Remote - SSH" Extension:**
    - In VS Code, open the Extensions view (Ctrl+Shift+X).
    - Search for `Remote - SSH`.

- Install the official extension provided by **Microsoft**.

3. **Configure the SSH Connection File:**
   - Open the Command Palette (`Ctrl+Shift+P`).
   - Type `Remote-SSH: Open SSH Configuration File...` and select it.
   - Choose the standard `config` file located in your user's `.ssh` directory.
4. **Add Host Entry:** Add a new block of text to this file to define each VM connection.

```
# Read more about SSH config files: https://linux.die.net/man/5/ssh_config
Host alias
    HostName hostname
    User user

# The Bastion Host - This one has a public IP
# Google Cloud VM for Open5GS Core
Host gcp-open5gs-vm
    HostName 35.224.186.219
    User telcomaan
    IdentityFile C:\Users\vishalkumar.shaw\.ssh\id_rsa

# Google Cloud VM for OAI-NR-UE
Host gcp-oai-nr-ue-vm
    HostName 34.28.167.5
    User telcomaan
    IdentityFile C:\Users\vishalkumar.shaw\.ssh\id_rsa

# Google Cloud VM for OAI-DU0
Host gcp-oai-du0-vm
    HostName 34.55.42.213
    User telcomaan
    IdentityFile C:\Users\vishalkumar.shaw\.ssh\id_rsa

# Google Cloud VM for OAI-CU
Host gcp-oai-cu-vm
    HostName 172.17.0.93
    User telcomaan
    ProxyJump gcp-open5gs-vm
    IdentityFile C:\Users\vishalkumar.shaw\.ssh\id_rsa

# Google Cloud VM for OAI-DU1
Host gcp-oai-du1-vm
```

```
    HostName 34.31.217.164
    User telcomaan
    IdentityFile C:\Users\vishalkumar.shaw\.ssh\id_rsa



# the HostName is the IP address of the VM (its epemeral and changes everytime
you start the VM)
# the User is the username you use to log in to the VM
# the IdentityFile is the path to your private SSH key file
```

5. **Save the `config` file.** Repeat Step 4 for any other VMs you wish to connect to, giving each a unique `Host` name and its correct `HostName` (External IP).
6. **Connect to the VM:**
   - Open the Command Palette (`Ctrl+Shift+P`).
   - Type `Remote-SSH: Connect to Host...` and select it.
   - Choose the host you configured (e.g., `gcp-open5gs-vm`) from the list.
   - A new VS Code window will open. You will be prompted for your **SSH key passphrase**. Enter it to complete the connection.
   - If the SSH connection fails , its likely due to conflict of hostname IP with the store list of previously used IPs. Check the `C:\Users\vishalkumar.shaw\.ssh\known_hosts` and remove the conflicting entries.

You are now successfully connected. The VS Code terminal and file explorer are directly linked to your remote GCP machine.

Part 4: Open5GS Core Network Installation & Configuration

SSH into the `open5gs-vm` and follow these steps.

1. **Install Prerequisites:**

```
sudo apt update
sudo apt install nano iptables git
sudo apt install -y software-properties-common curl gnupg git
```

2. **Install MongoDB:**

```
curl -fsSL https://pgp.mongodb.com/server-8.0.asc | sudo gpg -o
/usr/share/keyrings/mongodb-server-8.0.gpg --dearmor
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-
8.0.gpg] https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0
multiverse" | sudo tee /etc/apt/sources.list.d/mongodb-org-8.0.list
sudo apt update
sudo apt install -y mongodb-org
sudo systemctl start mongod
sudo systemctl enable mongod
```

3. **Install Open5GS:**

```
sudo add-apt-repository ppa:open5gs/latest
sudo apt update
sudo apt install open5gs -y
```

4. **Configure Open5GS Components:**
   Update the configuration files with the new IP address of your `open5gs-vm`.

- **In `/etc/open5gs/amf.yaml`:**

```
#...
amf:
  sbi: #...
  ngap:
    server:
      - address: 172.17.0.95 # open5gs vm ip
#...


# restart amf services
sudo systemctl restart open5gs-amfd

# amf logs can be found in /var/log/open5gs/amf.log
sudo tail -f /var/log/open5gs/amf.log
```

- **In `/etc/open5gs/upf.yaml`:**

```
#...
upf:
  pfcp: #...
```

```
  gtpu:
    server:
      - address: 172.17.0.95 # open5gs vm ip
#...


# restart upf services
sudo systemctl restart open5gs-upfd

# upf logs can be found in /var/log/open5gs/upf.log
sudo tail -f /var/log/open5gs/upf.log
```

- The `smf.yaml` and `nssf.yaml` files do not need IP changes for this basic setup. The default slice configurations are sufficient to start.

5. **Enable NAT:**

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING -o ens4 -j MASQUERADE
sudo iptables -I FORWARD 1 -j ACCEPT
```

6. **Setup and Add Subscriber in WebUI:**

- **Install Node.js and dependencies:**

```
sudo apt update
sudo apt install curl
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install nodejs -y
```

- **Clone the WebUI repo and install packages:**

```
cd ~
git clone https://github.com/open5gs/open5gs
cd open5gs/webui
npm install
```

- **Start the WebUI:**
  This command will run the user interface in the foreground.

```
npm run dev
```

- **Accessing the WebUI:**
  - VS-Code by default does port-forwarding and opens in on your localhost. Or you can do it other way given below.
  - The WebUI is now running on `localhost:3000` inside the VM. To access it from your computer, open a **new local terminal** (not the browser SSH) and run:

    ```
    gcloud compute ssh open5gs-vm --project=YOUR_PROJECT_ID --zone=us-central1-a -- -L 3000:localhost:3000
    ```

  - Now, open a browser on your local machine and go to `http://localhost:3000`.
  - Login with default credentials: `admin` / `1423`.
  - Add a new subscriber as per the document's instructions (IMSI, Key, OPC).
    - IMSI: 999700000000001
    - Subscriber Key: 465B5CE8B199B49FAA5F0A2EE238A6BC
    - USIM Type: OPc
    - Operator Key: E8ED289DEBA952E4283B54E88E6183CA

Part 5: OAI RAN Installation & Re-Building with Telnet

On **each of the four OAI VMs** (`oai-cu-vm`, `oai-du0-vm`, `oai-du1-vm`, `oai-nr-ue-vm`), perform the following steps. This build enables the telnet server required to trigger the handover.

```
# Run on all four OAI VMs
sudo apt update && sudo apt install -y git
cd ~
git clone https://gitlab.eurecom.fr/oai/openairinterface5g.git
cd openairinterface5g
source oaienv
cd cmake_targets
# This command builds all necessary components with telnet support
./build_oai --ninja --nrUE --gNB --build-lib telnetsrv
```

Part 6: OAI RAN Configuration for Handover

Create the configuration files in `/etc/oai/` on their respective VMs.

6.1. On `oai-cu-vm` (Consolidated CU):

- Create `/etc/oai/oai-cu.conf`

```
Active_gNBs = ( "gNB-Eurecom-CU");
# Asn1_verbosity, choice in: none, info, annoying
Asn1_verbosity = "none";

gNBs =
(
 {
    ////////// Identification parameters:
    gNB_ID = 0xe00;

#     cell_type =  "CELL_MACRO_GNB";

    gNB_name  =  "gNB-Eurecom-CU";

    // Tracking area code, 0x0000 and 0xfffe are reserved values
    tracking_area_code  =  1;
    plmn_list = ({ mcc = 999; mnc = 70; mnc_length = 2; snssaiList = ({ sst =
1, sd = 0x000001 }, { sst = 1, sd = 0xFFFFFF }, { sst = 2, sd = 0x111111 })
});

    nr_cellid = 12345678L;

    tr_s_preference = "f1";

    local_s_address = "172.17.0.93"; // cu-vm-ip
    remote_s_address = "172.17.0.92"; // du0-vm-ip
    local_s_portc   = 501;
    local_s_portd   = 2153;
    remote_s_portc  = 500;
    remote_s_portd  = 2153;

    # ------- SCTP definitions
    SCTP :
    {
        # Number of streams to use in input/output
        SCTP_INSTREAMS  = 5;
        SCTP_OUTSTREAMS = 5;
    };


    ////////// AMF parameters:
    amf_ip_address = ({ ipv4 = "172.17.0.95"; }); // open5gs-vm-ip
```

```
    NETWORK_INTERFACES :
    {

        GNB_IPV4_ADDRESS_FOR_NG_AMF              = "172.17.0.93"; // cu-vm-ip
        GNB_IPV4_ADDRESS_FOR_NGU                 = "172.17.0.93"; // cu-vm-ip
        GNB_PORT_FOR_S1U                         = 2152; # Spec 2152
    };
  }
);

security = {
  # preferred ciphering algorithms
  # the first one of the list that an UE supports in chosen
  # valid values: nea0, nea1, nea2, nea3
  ciphering_algorithms = ( "nea0" );

  # preferred integrity algorithms
  # the first one of the list that an UE supports in chosen
  # valid values: nia0, nia1, nia2, nia3
  integrity_algorithms = ( "nia2", "nia0" );

  # setting 'drb_ciphering' to "no" disables ciphering for DRBs, no matter
  # what 'ciphering_algorithms' configures; same thing for 'drb_integrity'
  drb_ciphering = "yes";
  drb_integrity = "no";
};
     log_config :
     {
       global_log_level                      ="info";
       hw_log_level                          ="info";
       phy_log_level                         ="info";
       mac_log_level                         ="info";
       rlc_log_level                         ="debug";
       pdcp_log_level                        ="info";
       rrc_log_level                         ="info";
       f1ap_log_level                         ="debug";
       ngap_log_level                         ="debug";
    };
```

6.2. On `oai-du0-vm` (Source DU0):

- Create `/etc/oai/oai-du0.conf`. Use the DU0 configuration (with `physCellId = 0`).

- **Crucially, modify the `rfsimulator` block** to configure this DU as a client pointing to the UE's IP address.

```
Active_gNBs = ( "oai-cu-cp");
Asn1_verbosity = "info";

gNBs =
(
 {
    gNB_ID = 0xe00;
    gNB_DU_ID = 0xe01;
    gNB_name  =  "oai-cu-cp";
    tracking_area_code  =  1;
    plmn_list = ({ mcc = 999; mnc = 70; mnc_length = 2; snssaiList =  ({ sst =
1, sd = 0x000001 },{ sst = 1, sd = 0xFFFFFF }) });
    nr_cellid = 12345678L;
    min_rxtxtime = 6;

    servingCellConfigCommon = (
    {
      physCellId = 0;
      absoluteFrequencySSB = 641280;
      dl_frequencyBand = 78;
      dl_absoluteFrequencyPointA = 640008;
      dl_offstToCarrier = 0;
      dl_subcarrierSpacing = 1;
      dl_carrierBandwidth = 106;
      initialDLBWPlocationAndBandwidth = 28875;
      initialDLBWPsubcarrierSpacing = 1;
      initialDLBWPcontrolResourceSetZero = 12;
      initialDLBWPsearchSpaceZero = 0;
      ul_frequencyBand = 78;
      ul_offstToCarrier = 0;
      ul_subcarrierSpacing = 1;
      ul_carrierBandwidth = 106;
      pMax = 20;
      initialULBWPlocationAndBandwidth = 28875;
      initialULBWPsubcarrierSpacing = 1;
      prach_ConfigurationIndex = 98;
      prach_msg1_FDM = 0;
      prach_msg1_FrequencyStart = 0;
      zeroCorrelationZoneConfig = 13;
      preambleReceivedTargetPower = -96;
      preambleTransMax = 6;
      powerRampingStep = 1;
      ra_ResponseWindow = 4;
```

```
      ssb_perRACH_OccasionAndCB_PreamblesPerSSB_PR = 4;
      ssb_perRACH_OccasionAndCB_PreamblesPerSSB = 14;
      ra_ContentionResolutionTimer = 7;
      rsrp_ThresholdSSB = 19;
      prach_RootSequenceIndex_PR = 2;
      prach_RootSequenceIndex = 1;
      msg1_SubcarrierSpacing = 1,
      restrictedSetConfig = 0,
      msg3_DeltaPreamble = 1;
      p0_NominalWithGrant = -90;
      pucchGroupHopping = 0;
      hoppingId = 40;
      p0_nominal = -90;
      ssb_PositionsInBurst_Bitmap = 1;
      ssb_periodicityServingCell = 2;
      dmrs_TypeA_Position = 0;
      subcarrierSpacing = 1;
      referenceSubcarrierSpacing = 1;
      dl_UL_TransmissionPeriodicity = 6;
      nrofDownlinkSlots = 7;
      nrofDownlinkSymbols = 6;
      nrofUplinkSlots = 2;
      nrofUplinkSymbols = 4;
      ssPBCH_BlockPower = -25;
     }
  );

    SCTP :
    {
        SCTP_INSTREAMS  = 2;
        SCTP_OUTSTREAMS = 2;
    };
  }
);

MACRLCs = (
  {
    num_cc           = 1;
    tr_s_preference  = "local_L1";
    tr_n_preference  = "f1";
    local_n_address = "172.17.0.92";
    remote_n_address = "172.17.0.93";
    local_n_portc   = 500;
    local_n_portd   = 2153;
    remote_n_portc  = 38472;
    remote_n_portd  = 2153;
```

```
    pusch_TargetSNRx10          = 200;
    pucch_TargetSNRx10          = 200;
  }
);

L1s = (
{
  num_cc = 1;
  tr_n_preference = "local_mac";
  prach_dtx_threshold = 200;
  pucch0_dtx_threshold = 150;
  ofdm_offset_divisor = 8;
}
);

RUs = (
    {
       local_rf       = "yes"
         nb_tx          = 1
         nb_rx          = 1
         att_tx         = 0
         att_rx         = 0;
         bands          = [78];
         max_pdschReferenceSignalPower = -27;
         max_rxgain                    = 114;
         eNB_instances  = [0];
         clock_src = "internal";
    }
);

rfsimulator: {
serveraddr = "172.17.0.91";
    serverport = 4043;
    options = ();
    modelname = "AWGN";
    IQfile = "/tmp/rfsimulator.iqs"
}

log_config: {
  global_log_level = "info";
  hw_log_level = "info";
  phy_log_level = "info";
  mac_log_level = "info";
  rlc_log_level = "info";
  f1ap_log_level = "info";
};
```

```
channelmod = {
  max_chan = 10;
  modellist = "modellist_rfsimu_1";
  modellist_rfsimu_1 = (
    {
      model_name     = "rfsimu_channel_enB0"
      type           = "AWGN";
      ploss_dB       = 20;
      noise_power_dB = -4;
      forgetfact     = 0;
      offset         = 0;
      ds_tdl         = 0;
    },
    {
      model_name     = "rfsimu_channel_ue0"
      type           = "AWGN";
      ploss_dB       = 20;
      noise_power_dB = -2;
      forgetfact     = 0;
      offset         = 0;
      ds_tdl         = 0;
    }
  );
};
```

6.3. On `oai-du1-vm` (Source DU0):

- Create `/etc/oai/oai-du1.conf`. Use the DU1 configuration (with gNB_DU_ID = 0xe02 and physCellId = 1).
- **Crucially, modify the `rfsimulator` block** to configure this DU as a client pointing to the UE's IP address.

```
Active_gNBs = ( "oai-cu-cp" );
Asn1_verbosity = "info";

gNBs =
(
 {
    gNB_ID = 0xe00;
    gNB_DU_ID = 0xe02;
    gNB_name = "oai-cu-cp";
    tracking_area_code = 1;
```

```
    plmn_list = (
      {
        mcc = 999;
        mnc = 70;
        mnc_length = 2;
        snssaiList = (
          { sst = 1; sd = 0x000001; },
          { sst = 1; sd = 0xFFFFFF; }
        );
      }
    );
    nr_cellid = 12345679L;
    min_rxtxtime = 6;

    servingCellConfigCommon = (
      {
        physCellId = 1;
        absoluteFrequencySSB = 641280;
        dl_frequencyBand = 78;
        dl_absoluteFrequencyPointA = 640008;
        dl_offstToCarrier = 0;
        dl_subcarrierSpacing = 1;
        dl_carrierBandwidth = 106;
        initialDLBWPlocationAndBandwidth = 28875;
        initialDLBWPsubcarrierSpacing = 1;
        initialDLBWPcontrolResourceSetZero = 12;
        initialDLBWPsearchSpaceZero = 0;
        ul_frequencyBand = 78;
        ul_offstToCarrier = 0;
        ul_subcarrierSpacing = 1;
        ul_carrierBandwidth = 106;
        pMax = 20;
        initialULBWPlocationAndBandwidth = 28875;
        initialULBWPsubcarrierSpacing = 1;
        prach_ConfigurationIndex = 98;
        prach_msg1_FDM = 0;
        prach_msg1_FrequencyStart = 0;
        zeroCorrelationZoneConfig = 13;
        preambleReceivedTargetPower = -96;
        preambleTransMax = 6;
        powerRampingStep = 1;
        ra_ResponseWindow = 4;
        ssb_perRACH_OccasionAndCB_PreamblesPerSSB_PR = 4;
        ssb_perRACH_OccasionAndCB_PreamblesPerSSB = 14;
        ra_ContentionResolutionTimer = 7;
        rsrp_ThresholdSSB = 19;
```

```
        prach_RootSequenceIndex_PR = 2;
        prach_RootSequenceIndex = 1;
        msg1_SubcarrierSpacing = 1;
        restrictedSetConfig = 0;
        msg3_DeltaPreamble = 1;
        p0_NominalWithGrant = -90;
        pucchGroupHopping = 0;
        hoppingId = 40;
        p0_nominal = -90;
        ssb_PositionsInBurst_Bitmap = 1;
        ssb_periodicityServingCell = 2;
        dmrs_TypeA_Position = 0;
        subcarrierSpacing = 1;
        referenceSubcarrierSpacing = 1;
        dl_UL_TransmissionPeriodicity = 6;
        nrofDownlinkSlots = 7;
        nrofDownlinkSymbols = 6;
        nrofUplinkSlots = 2;
        nrofUplinkSymbols = 4;
        ssPBCH_BlockPower = -25;
      }
    );

    SCTP =
    {
        SCTP_INSTREAMS = 2;
        SCTP_OUTSTREAMS = 2;
    };
  }
);

MACRLCs = (
  {
    num_cc = 1;
    tr_s_preference = "local_L1";
    tr_n_preference = "f1";
    local_n_address = "172.17.0.94";
    remote_n_address = "172.17.0.93";
    local_n_portc = 500;
    local_n_portd = 2153;
    remote_n_portc = 38472;
    remote_n_portd = 2153;
    pusch_TargetSNRx10 = 200;
    pucch_TargetSNRx10 = 200;
  }
);
```

```
L1s = (
  {
    num_cc = 1;
    tr_n_preference = "local_mac";
    prach_dtx_threshold = 200;
    pucch0_dtx_threshold = 150;
    ofdm_offset_divisor = 8;
  }
);

RUs = (
  {
    local_rf = "yes";
    nb_tx = 1;
    nb_rx = 1;
    att_tx = 0;
    att_rx = 0;
    bands = [78];
    max_pdschReferenceSignalPower = -27;
    max_rxgain = 114;
    eNB_instances = [0];
    clock_src = "internal";
  }
);

rfsimulator =
{
  serveraddr = "172.17.0.91";
  serverport = 4043;
  options = ();
  modelname = "AWGN";
  IQfile = "/tmp/rfsimulator.iqs";
};

log_config =
{
  global_log_level = "info";
  hw_log_level = "info";
  phy_log_level = "info";
  mac_log_level = "info";
  rlc_log_level = "info";
  f1ap_log_level = "info";
};

channelmod =
```

```
{
  max_chan = 10;
  modellist = "modellist_rfsimu_1";
  modellist_rfsimu_1 = (
    {
      model_name = "rfsimu_channel_enB0";
      type = "AWGN";
      ploss_dB = 20;
      noise_power_dB = -4;
      forgetfact = 0;
      offset = 0;
      ds_tdl = 0;
    },
    {
      model_name = "rfsimu_channel_ue0";
      type = "AWGN";
      ploss_dB = 20;
      noise_power_dB = -2;
      forgetfact = 0;
      offset = 0;
      ds_tdl = 0;
    }
  );
};
```

6.4. On `oai-nr-ue-vm` (UE):

- Create `/etc/oai/nr-ue.conf` with only the subscriber's UICC information. with the `rfsimulator` block , and declare it as the rf-server.

```
uicc0 = {
  imsi = "999700000000001";
  key = "465B5CE8B199B49FAA5F0A2EE238A6BC";
  opc = "E8ED289DEBA952E4283B54E88E6183CA";
  dnn = "internet";
  nssai_sst = 1;
  nssai_sd = 0xFFFFFF;
}

rfsimulator: {
serveraddr = "server";
    serverport = 4043;
    options = (); #("saviq"); or/and "chanmod"
    modelname = "AWGN";
```

```
    IQfile = "/tmp/rfsimulator.iqs"
}
```

6.5. The RF Simulator: UE as Server for Handover

For a standard single-cell setup, it is common to configure the gNB as the RF simulator "server" and the UE as the "client." However, the F1 handover scenario introduces a critical complexity that requires this model to be inverted.

The Challenge: Connecting to Two DUs

During handover, the UE must be able to listen to the source DU (DU0) and then seamlessly switch to listen to the target DU (DU1). If both DUs were configured as independent RF simulator servers, the UE client would only be able to connect to one at a time. It would have no way of discovering or hearing the transmission from the second DU's simulation environment, causing the handover to fail at the physical layer (as seen by a `synch Failed` error in the UE logs).

The Solution: A Centralized, UE-Hosted Simulation

The OAI RF simulator (`--rfsim`) operates on a strict **one-server-to-many-clients** model for any given simulation instance. To solve the handover problem, the component that needs to communicate with multiple peers must become the server. In this case, that component is the UE.

By configuring the UE as the server, we create a single, centralized simulation environment. Both the source DU and the target DU then act as clients that connect to this UE-hosted environment.

This "hub-and-spoke" architecture ensures that:

- The signals from **both** DUs are being fed into the *same* simulation instance.
- When the UE receives the handover command, it can immediately start listening for the target DU's signals within the simulation environment it is already hosting.

How This is Implemented:

- **The UE (`nr-uesoftmodem`) as the Server:**
  The UE is configured as the server through its startup command. By including the `--rfsim` flag *without* the `--rfsimulator.serveraddr` argument, it defaults to server mode. And it is explicitly configured as rf server in the conf file.

```
rfsimulator: {
    serveraddr = "server";
    ...
}
```

- **The DUs (`nr-softmodem`) as Clients:**
  Both DU configuration files (`oai-du0.conf` and `oai-du1.conf`) are explicitly configured as clients by setting the `serveraddr` parameter in the `rfsimulator` block to the UE's IP address:

```
rfsimulator: {
    serveraddr = "172.17.0.91"; # <-- The IP of the UE acting as the server
    ...
}
```

This inverted server-client model is the key to enabling a successful RF-simulated F1 handover in OAI.

Part 7: Starting the Network & Executing the F1 Handover

This startup sequence is critical and must be followed exactly. Use a separate SSH terminal for each command.

1. Start Open5GS Services (on `open5gs-vm`)

```
# MongoDB (if using local database)
sudo systemctl start mongod

# Start Open5GS core components
sudo systemctl start open5gs-mmed
sudo systemctl start open5gs-sgwcd
sudo systemctl start open5gs-smfd
sudo systemctl start open5gs-amfd
sudo systemctl start open5gs-upfd
sudo systemctl start open5gs-pcfd
sudo systemctl start open5gs-ausfd
sudo systemctl start open5gs-nrfd
sudo systemctl start open5gs-nssfd
sudo systemctl start open5gs-bsfd
```

```
sudo systemctl start open5gs-udmd
sudo systemctl start open5gs-udrd
sudo systemctl start open5gs-pcfd

# Verify that all services are running
sudo systemctl status open5gs-*
```

2. Enable NAT (on `open5gs-vm`)

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING -o ens4 -j MASQUERADE
sudo iptables -I FORWARD 1 -j ACCEPT
```

3. Start the CU (on `oai-cu-vm`)

```
# On: oai-cucp-vm
cd ~/openairinterface5g/cmake_targets/ran_build/build
sudo -E ./nr-softmodem -O /etc/oai/oai-cu.conf --sa --telnetsrv --
telnetsrv.shrmod ci
```

5. Start the NR UE as the RF Server (on `oai-nr-ue-vm`)

This starts the UE in server mode, waiting for the DUs to connect.

```
# On: oai-nr-ue-vm
cd ~/openairinterface5g/cmake_targets/ran_build/build
sudo ./nr-uesoftmodem -r 106 --numerology 1 --band 78 -C 3619200000 --ssb 516
--rfsim -O /etc/oai/nr-ue.conf
```

6. Start the Source DU (DU0) as a Client (on `oai-du0-vm`)

The DU will connect to the UE's RF server, and the UE will attach to the network.

```
# On: oai-du-vm
cd ~/openairinterface5g/cmake_targets/ran_build/build
sudo ./nr-softmodem -O /etc/oai/oai-du0.conf --rfsim --sa
```

7. Start the Target DU (DU1) as a Client (on `oai-du1-vm`)

This DU also connects to the UE's RF server. The CU-CP log will show a second F1 setup.

```
# On: oai-cuup-vm
cd ~/openairinterface5g/cmake_targets/ran_build/build
```

```
sudo ./nr-softmodem -O /etc/oai/oai-du1.conf --rfsim --sa
```

8. Trigger the F1 Handover
Run this command from any machine.

```
echo ci trigger_f1_ho | nc 172.17.0.93 9090 && echo
```

Part 8: Verifying the Successful Handover

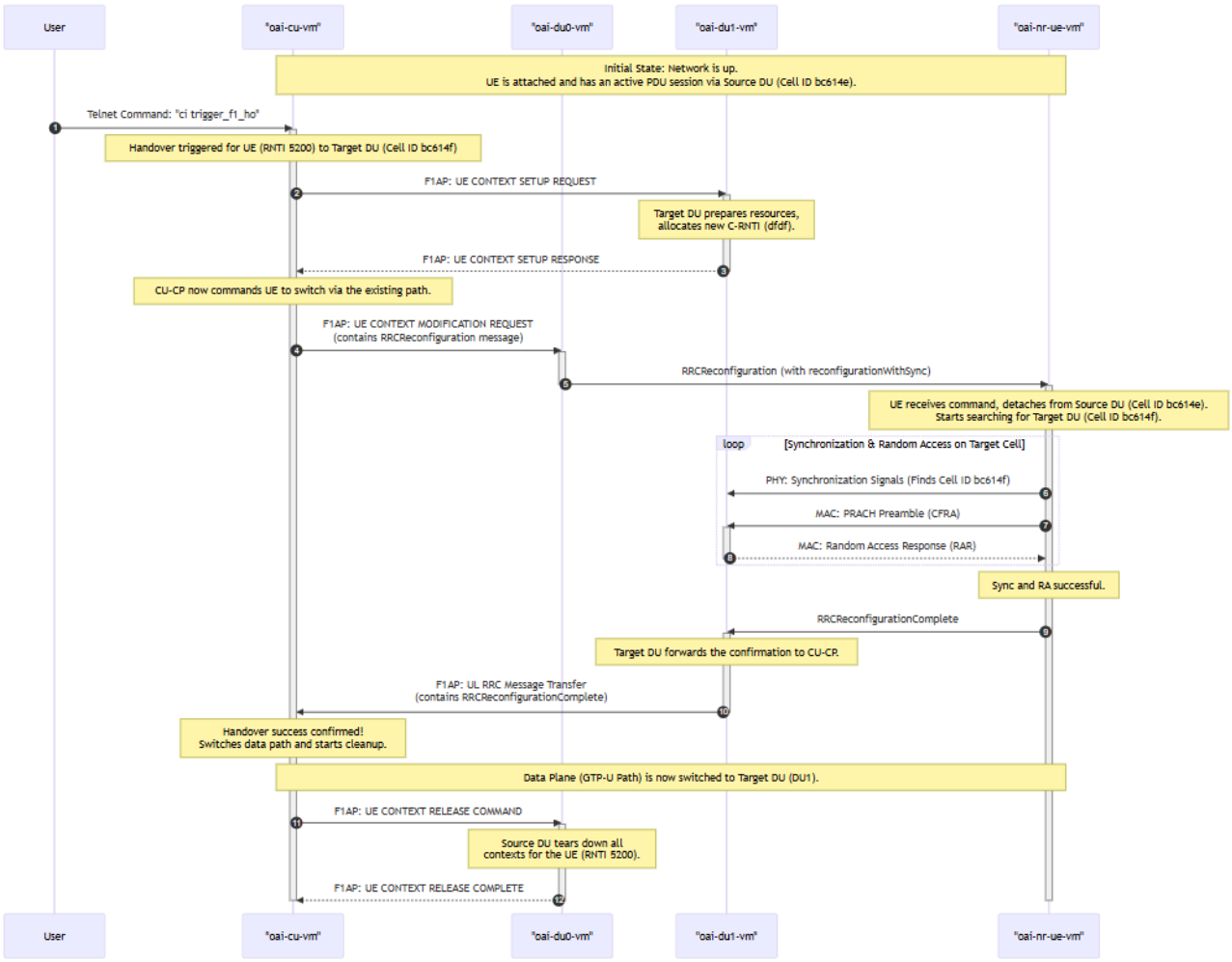
1. **Check UE Logs:** The UE log (`oai-nr-ue-vm`) will show it receiving `RRCReconfiguration` with `reconfigurationWithSync` , successfully synchronizing to the new cell (PCI 1), and sending `RRCReconfigurationComplete` .
2. **Check CU-CP Logs:** Will show the "Handover triggered" message.
3. **Check AMF Logs:** The AMF logs **will show no new activity**. This is correct and expected, as an F1 handover is contained within the RAN and is transparent to the core network.
4. **Test Data Connectivity:** The definitive test. On the `oai-nr-ue-vm` , a continuous ping should survive the handover.

   ```
   # On oai-nr-ue-vm, before, during, and after triggering the handover
   ping -I oaitun_ue1 8.8.8.8
   ```

   Successful, uninterrupted replies confirm a successful data plane switch.

Part 9:The F1 Handover Process Explained

This section breaks down the key stages of the F1 handover you triggered. By cross-referencing these log snippets with your own terminal windows, you can trace the entire procedure from start to finish.



Step 1: Network Ready State - UE Attached via Source DU (DU0)

Initially, the network is stable. The UE has attached to the core through the source DU (`oai-du0-vm` , PCI 0). Both DUs have established their F1 control connections with the CU-CP, and the CU-CP has established its E1 and NGAP connections.

- **Log: CU-CP (`oai-cu-vm`) shows all links are up and the UE is attached.**
  The log shows the NGAP setup with the AMF is complete. It then accepts F1 connections from both the source DU (ID `3585`) and the target DU (ID `3586`). Finally, it processes the UE's connection through the source DU, establishing a PDU session.

```
[NGAP]     Received NGSetupResponse from AMF
[GNB_APP]  [gNB 0] Received NGAP_REGISTER_GNB_CNF: associated AMF 1
...
[NR_RRC]   Received F1 Setup Request from gNB_DU 3585 (oai-cu-cp) on
assoc_id 6
[NR_RRC]   Accepting DU 3585 (oai-cu-cp), sending F1 Setup Response
...
```

```
[NR_RRC]   Received F1 Setup Request from gNB_DU 3586 (oai-cu-cp) on
assoc_id 7
[NR_RRC]   Accepting DU 3586 (oai-cu-cp), sending F1 Setup Response
...
[NR_RRC]   [--] (cellID 0, UE ID 1 RNTI 5200) Create UE context...
[NR_RRC]   [UL] (cellID bc614e, UE ID 1 RNTI 5200) Received
RRCSetupComplete (RRC_CONNECTED reached)
...
[NGAP]     PDUSESSIONSetup initiating message
[NR_RRC]   UE 1: received PDU Session Resource Setup Request
[NR_RRC]   [DL] (cellID bc614e, UE ID 1 RNTI 5200) Generate
RRCReconfiguration (bytes 283, xid 0)
[NR_RRC]   [UL] (cellID bc614e, UE ID 1 RNTI 5200) Received
RRCReconfigurationComplete
```

- **Log: UE (`oai-nr-ue-vm`) confirms synchronization and PDU session.**
  The UE's log shows it synchronizing to the initial cell (PCI 0), completing the full NAS registration and security procedures, and finally receiving its IP address, which activates the `oaitun_ue1` interface.

```
[PHY]      Initial sync successful, PCI: 0
...
[NR_RRC]   State = NR_RRC_CONNECTED
[NAS]      Generate Initial NAS Message: Registration Request
...
[NAS]      [UE 0] Received NAS_DOWNLINK_DATA_IND type
FGS_REGISTRATION_ACCEPT with length 46
...
[NAS]      Received PDU Session Establishment Accept, UE IPv4: 10.45.0.2
[OIP]      Interface oaitun_ue1 successfully configured, IPv4 10.45.0.2,
IPv6 (null)
```

Step 2: Manual Handover Trigger

You send the `ci trigger_f1_ho` command to the telnet server on the CU-CP. This is the catalyst for the entire sequence.

- **Log: CU-CP (`oai-cu-vm`) receives the command and initiates the handover.**
  The CU-CP receives the command and immediately begins the handover procedure, targeting the UE (identified by its original RNTI `5200`) towards the target DU (assoc_id `7`, which corresponds to PCI `1`).

```
[TELNETSRV] Telnet client connected....
[TELNETSRV] Command received: readc 17 filled 17 "ci trigger_f1_ho"
[NR_RRC]   Handover triggered for UE 1/RNTI 5200 towards DU 3586/assoc_id
7/PCI 1
```

Step 3: Handover Preparation & RRC Command

The CU-CP orchestrates the move. It first notifies the target DU to prepare resources. Once confirmed, it sends the `RRCReconfiguration` message to the UE, telling it to switch.

- **Log: Target DU (`oai-du1-vm`) prepares for the UE's arrival.**
  The target DU receives an F1AP message from the CU-CP (not explicitly shown but implied). It sets up a new context for the incoming UE, assigning it a new temporary identifier (C-RNTI `dfdf`) and creating all necessary radio and transport layers for it.

```
[NR_MAC]   Added new CFRA process for UE RNTI dfdf with initial CellGroup
[RLC]      Activated srb0 for UE 57311
[RLC]      Added srb 1 to UE 57311
[RLC]      Added srb 2 to UE 57311
[RLC]      Added drb 1 to UE 57311
[GTPU]     [94] Created tunnel for UE ID 57311, teid for incoming:
65e56dc2...
```

- **Log: UE (`oai-nr-ue-vm`) receives the handover command and starts the process.**
  The UE receives the `RRCReconfiguration` message. The key fields `reconfigurationWithSync` and the new C-RNTI `dfdf` instruct it to perform a handover.

```
[NR_RRC]   RRCReconfiguration includes radio Bearer Configuration
[PDCP]     SRB 2 re-established
[PDCP]     DRB 1 re-established
[NR_RRC]   State = NR_RRC_CONNECTED
[NR_RRC]   Processing reconfigurationWithSync
...
[MAC]      [UE 0] Applying CellGroupConfig from gNodeB
[NR_MAC]   Received reconfigurationWithSync
[NR_MAC]   Configuring CRNTI dfdf
```

Step 4: UE Re-synchronization to Target Cell (PCI 1)

This is the most critical phase where the UE's physical layer finds and locks onto the new cell.

- **Log: UE (`oai-nr-ue-vm`) successfully finds and synchronizes to PCI 1.**
  The UE's PHY layer immediately starts searching for the new cell ID (`Nid_cell 1`) and successfully decodes its broadcast channel, confirming synchronization.

```
[NR_PHY]   Starting re-sync detection for target Nid_cell 1
[PHY]      [UE thread Synch] Running Initial Synch
...
[PHY]      Initial sync: pbch decoded sucessfully, ssb index 0
[PHY]      pbch rx ok. rsrp:54 dB/RE, adjust_rxgain:-4 dB
[NR_PHY]   Cell Detected with GSCN: 0...
[PHY]      Initial sync successful, PCI: 1
[PHY]      Got synch: hw_slot_offset 24, carrier off -186 Hz...
```

Step 5: Random Access on Target DU and Handover Completion

After synchronizing, the UE performs a contention-free random access (CFRA) on the new cell to announce its arrival and sends the `RRCReconfigurationComplete` message to finalize the handover.

- **Log: Target DU (`oai-du1-vm`) detects the UE's PRACH and completes the access.**
  The target DU sees the PRACH preamble from the UE, responds, and logs the successful completion of the contention-free random access with the UE's new RNTI `dfdf` .

```
[NR_PHY]   [RAPROC] 563.19 Initiating RA procedure with preamble 63, energy
44.0 dB...
[NR_MAC]   563.19 UE RA-RNTI 0113 TC-RNTI dfdf: initiating RA procedure
...
[NR_MAC]   (rnti 0xdfdf) CFRA procedure succeeded!
[NR_MAC]   Adding new UE context with RNTI 0xdfdf
```

- **Log: UE (`oai-nr-ue-vm`) sends the final confirmation.**
  The UE's MAC layer confirms the random access succeeded, and the RRC layer generates the `RRCReconfigurationComplete` message to send up to the network.

```
[MAC]      [UE 0][564.7][RAPROC] RA procedure succeeded. CFRA: RAR
successfully received.
...
[NR_RRC]   rrcReconfigurationComplete Encoded 10 bits (2 bytes)
```

```
[NR_RRC]   Logical Channel UL-DCCH (SRB1), Generating
RRCReconfigurationComplete (bytes 2)
```

Step 6: Network Path Switch and Resource Cleanup

The CU-CP, having received the confirmation from the UE via the target DU, now finalizes the handover by switching the data path and telling the source DU to clean up.

- **Log: Source DU (`oai-du0-vm`) is commanded to release the UE's context.**
  The source DU receives a `TransmissionActionIndicator with Stop value` for the original UE RNTI `5200`. After a timer, it tears down all resources associated with that UE.

```
[NR_MAC]   gNB-DU received the TransmissionActionIndicator with Stop value
for UE 5200
...
[GTPU]     [94] Deleted all tunnels for ue id 20992 (1 tunnels deleted)
[RLC]      Remove UE 20992
[NR_MAC]   Remove NR rnti 0x5200
```

- **Log: CU-CP (`oai-cu-vm`) logs the final success and triggers the cleanup.**
  The CU-CP receives the `RRCReconfigurationComplete` via the new path (target DU), updates the UE's RNTI from `5200` to `dfdf`, and declares the handover complete. It then explicitly triggers the release of the UE context on the source DU (assoc_id `6`).

```
[NR_RRC]   UE 1 handover: update RNTI from 5200 to dfdf
[NR_RRC]   [UL] (cellID bc614f, UE ID 1 RNTI dfdf) Received
RRCReconfigurationComplete
[NR_RRC]   handover for UE 1/RNTI dfdf complete!
[NR_RRC]   UE 1 Handover: trigger release on DU assoc_id 6
```

At this point, the procedure is complete. The UE is actively communicating through the target DU, and the data from the continuous ping test is flowing seamlessly through the new data path.

Part 10: CRITICAL Cleanup

To avoid ANY charges after you are finished, you **MUST** tear down your environment.

1. **Delete the VMs:**

- Go to **Compute Engine > VM instances**.
- Select all five VMs and click **DELETE**.

2. **Delete the Cloud NAT Gateway:**
   - Go to **Network services > Cloud NAT**.
   - Select the `oai-5g-nat` gateway and click **DELETE**.
3. **Delete VPC, Firewall Rules, and Static IPs:**
   - Optionally, delete these components to leave your project completely clean.

Links

- F1-Handover procedure draw.io --> https://drive.google.com/file/d/1YerukXAROqbL3o8jTNwJxVJxzveAqrha/view?usp=sharing
- F1-Handover for rf-sim setup draw.io --> https://drive.google.com/file/d/1wpv_xx1RJUNTTTLFXiSuFpAVwUbzWNB_/view?usp=sharing
- Detailed log files -->
- Demo video -->